Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра инженерной психологии и эргономики
Учебная дисциплина «Метрология, стандартизация и сертификация (в информационных технологиях)»

ОТЧЁТ

по лабораторной работе № 2б
«Метрика граничных значений»

Подготовили студенты гр. 410902
Большакова М. В.
Барбашёва А. А.

Проверила Болтак С. В.

Минск 2026

Язык: C++.

Задание:
Для подготовленной программы на языке C++ определить метрику граничных значений: выделить подграфы для вершин выбора, вычислить скорректированную сложность вершин, абсолютную и относительную сложности. Схема алгоритма приведена в приложении А, исходный код — в приложении Б.

Результаты расчёта

Таблица 1 – Свойства подграфов программы

Функция	Вершина выбора	Переходы	Вершины до границы	Граница	Сложность
main()	3	4, 5	4–9	10	7
	5	6, 7	6–9	10	5
	7	8, 9	8, 9	10	3
	11	12, 13	12, 13	14	3
	20	21, 24	20–23	24	5
	28	29, 31	28–30	31	4
	36	35, 37	35, 36	37	3
	40	41, 43	40–42	43	4
	46	47, 56	46–55	56	11
	48	49, 55	48–54	55	8
	49	50, 51	48–54	55	8
	51	52, 53	48–54	55	8
	64	65, 66	65, 66	67	3
	75	76, 78	76–78	79	4
	80	81, 84	80–83	84	5
factorial(n)	2	3, 4	3, 4	5	3
power(base, exp)	4	5, 7	4–6	7	4
isPositive(value)	2	3, 4	3, 4	5	3
colorName(c)	2	3, 4	3–8	9	7
	4	5, 6	5–8	9	5
	6	7, 8	7, 8	9	3

Скорректированная сложность вершин

Таблица 2 – Значения сложности

Функция	**Номера вершин**	**Скорректированная сложность**
main()	1–17	1, 1, 7, 1, 5, 1, 3, 1, 1, 1, 3, 1, 1, 1, 1, 1, 1
	18–34	1, 1, 5, 1, 1, 1, 1, 1, 1, 1, 4, 1, 1, 1, 1, 1, 1
	35–51	1, 3, 1, 1, 1, 4, 1, 1, 1, 1, 1, 11, 1, 8, 8, 1, 8
	52–68	1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1
	69–85	1, 1, 1, 1, 1, 1, 4, 1, 1, 1, 1, 5, 1, 1, 1, 1, 0
factorial(n)	1–5	1, 3, 1, 1, 0
square(a)	1–3	1, 1, 0
power(base, exp)	1–8	1, 1, 1, 4, 1, 1, 1, 0
movePoint(p, dx, dy)	1–4	1, 1, 1, 0
isPositive(value)	1–5	1, 3, 1, 1, 0
colorName(c)	1–9	1, 7, 1, 5, 1, 3, 1, 1, 0

Абсолютная сложность S_a равна сумме значений из таблицы 2. Для относительной сложности используем формулу $S_o = 1 - (V - 1) / S_a$, где V — общее число вершин схемы. Всего V = 119. Общая схема square(a) для двух перегрузок учитывается один раз. Сумма сложностей 21 вершины выбора равна 106; ещё 91 вершина имеет сложность 1, а 7 конечных вершин — 0.

$$S_a = 106 + 91 = 197.$$
$$S_o = 1 - (119 - 1) / 197 = 79 / 197 \approx 0{,}401.$$

Вывод

Абсолютная граничная сложность равна 197, относительная — 0,401. Наибольшая скорректированная сложность равна 11 и соответствует вершине 46 — условию внешнего цикла в main().

Приложение А Схема алгоритма

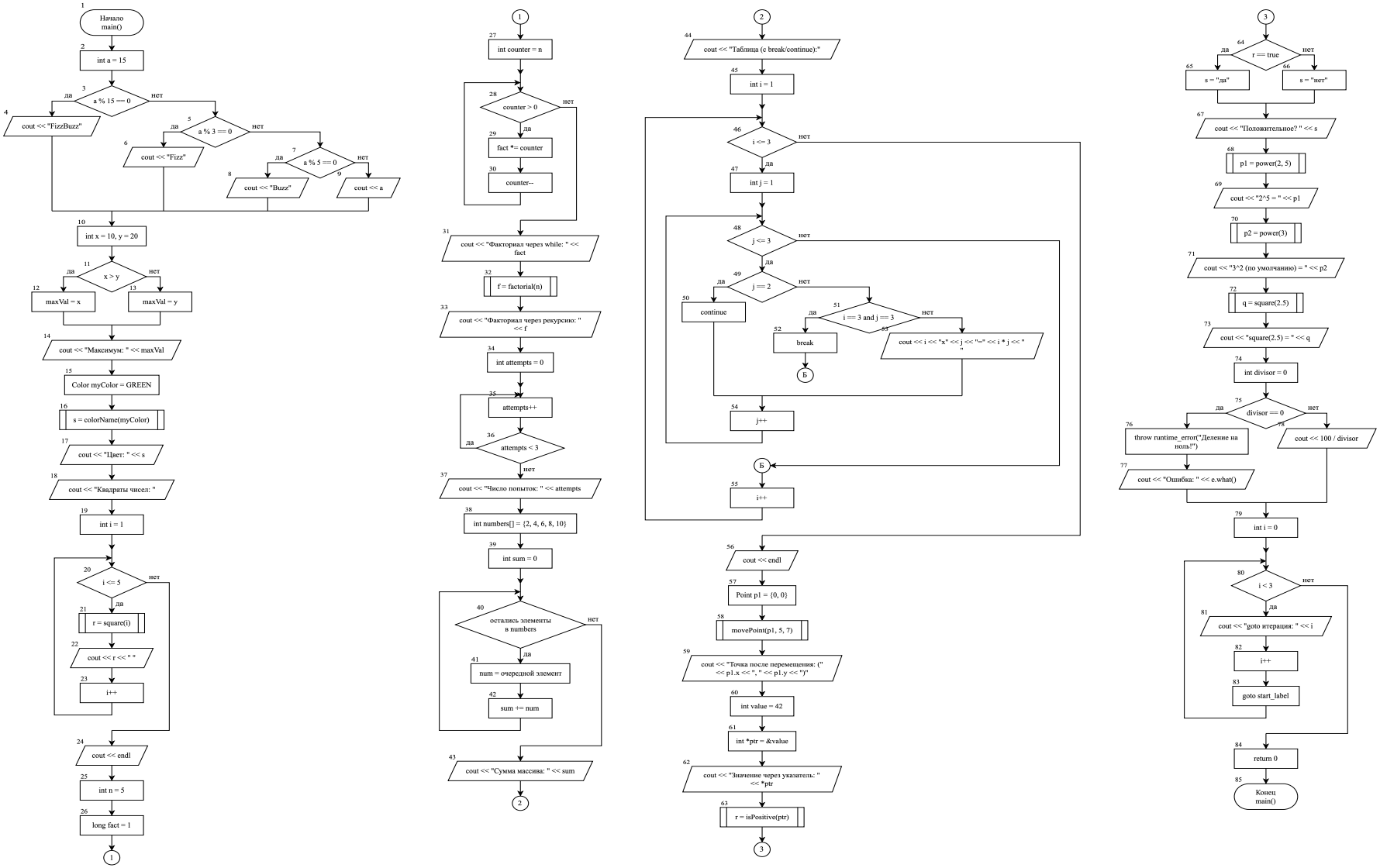


Рисунок А.1 – Схема main()

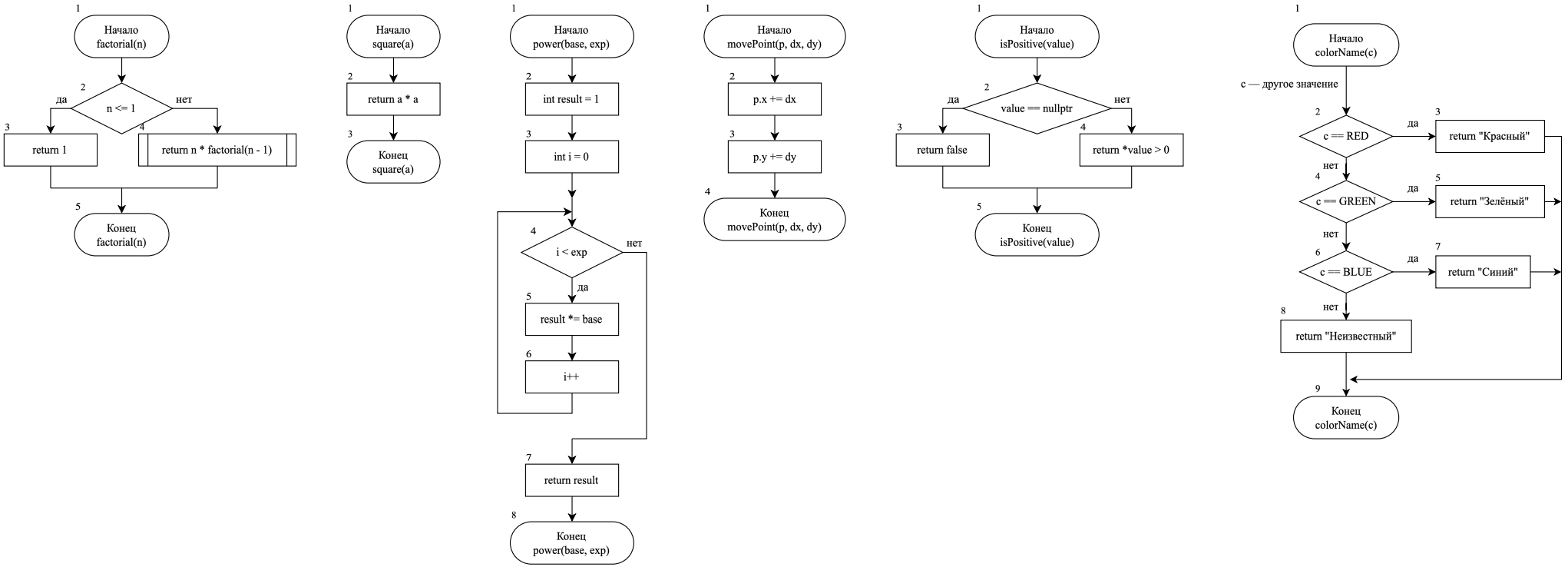


Рисунок А.2 – Схемы вспомогательных функций

Приложение Б Код проверяемой программы

```
#include <iostream>
#include <string>
using namespace std;

enum Color { RED, GREEN, BLUE };   // перечисление

struct Point {                      // структура
    int x;
    int y;
};

// Рекурсивная пользовательская функция
long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

// Перегрузка функции (overload) №1
int square(int a) { return a * a; }
// Перегрузка функции №2
double square(double a) { return a * a; }

// Функция с параметром по умолчанию
int power(int base, int exp = 2) {
    int result = 1;
    for (int i = 0; i < exp; i++) result *= base;
    return result;
}

// Функция, принимающая структуру по ссылке (изменяет исходные данные)
void movePoint(Point &p, int dx, int dy) {
    p.x += dx;
    p.y += dy;
}

// Функция с указателем и проверкой на nullptr
bool isPositive(const int *value) {
    if (value == nullptr) return false;
    return *value > 0;
}

// Функция, возвращающая название цвета (switch)
string colorName(Color c) {
    switch (c) {
        case RED:   return "Красный";
        case GREEN: return "Зелёный";
        case BLUE:  return "Синий";
        default:    return "Неизвестный";
    }
}
```

```
int main() {
    // --- if / else if / else ---
    int a = 15;
    if (a % 15 == 0) {
        cout << "FizzBuzz" << endl;
    } else if (a % 3 == 0) {
        cout << "Fizz" << endl;
    } else if (a % 5 == 0) {
        cout << "Buzz" << endl;
    } else {
        cout << a << endl;
    }

    // --- тернарный оператор ---
    int x = 10, y = 20;
    int maxVal = (x > y) ? x : y;
    cout << "Максимум: " << maxVal << endl;

    // --- switch ---
    Color myColor = GREEN;
    cout << "Цвет: " << colorName(myColor) << endl;

    // --- for ---
    cout << "Квадраты чисел: ";
    for (int i = 1; i <= 5; i++) {
        cout << square(i) << " ";
    }
    cout << endl;

    // --- while ---
    int n = 5;
    long fact = 1;
    int counter = n;
    while (counter > 0) {
        fact *= counter;
        counter--;
    }
    cout << "Факториал через while: " << fact << endl;
    cout << "Факториал через рекурсию: " << factorial(n) << endl;

    // --- do...while ---
    int attempts = 0;
    do {
        attempts++;
    } while (attempts < 3);
    cout << "Число попыток: " << attempts << endl;

    // --- range-based for (диапазонный цикл) ---
    int numbers[] = {2, 4, 6, 8, 10};
    int sum = 0;
    for (int num : numbers) {
        sum += num;
```

```
    }
    cout << "Сумма массива: " << sum << endl;

    // --- вложенный цикл + break/continue ---
    cout << "Таблица (с break/continue):" << endl;
    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            if (j == 2) continue;
            if (i == 3 && j == 3) break;
            cout << i << "x" << j << "=" << i * j << " ";
        }
    }
    cout << endl;

    // --- работа со структурой и передачей по ссылке ---
    Point p1 = {0, 0};
    movePoint(p1, 5, 7);
    cout << "Точка после перемещения: (" << p1.x << ", " << p1.y << ")" << endl;

    // --- указатели ---
    int value = 42;
    int *ptr = &value;
    cout << "Значение через указатель: " << *ptr << endl;
    cout << "Положительное? " << (isPositive(ptr) ? "да" : "нет") << endl;

    // --- параметр по умолчанию и перегрузка ---
    cout << "2^5 = " << power(2, 5) << endl;
    cout << "3^2 (по умолчанию) = " << power(3) << endl;
    cout << "square(2.5) = " << square(2.5) << endl;

    // --- try/catch (обработка исключений) ---
    try {
        int divisor = 0;
        if (divisor == 0) {
            throw runtime_error("Деление на ноль!");
        }
        cout << 100 / divisor << endl;
    } catch (const runtime_error &e) {
        cout << "Ошибка: " << e.what() << endl;
    }

    // --- goto ---
    int i = 0;
start_label:
    if (i < 3) {
        cout << "goto итерация: " << i << endl;
        i++;
        goto start_label;
    }

    return 0;
}
```